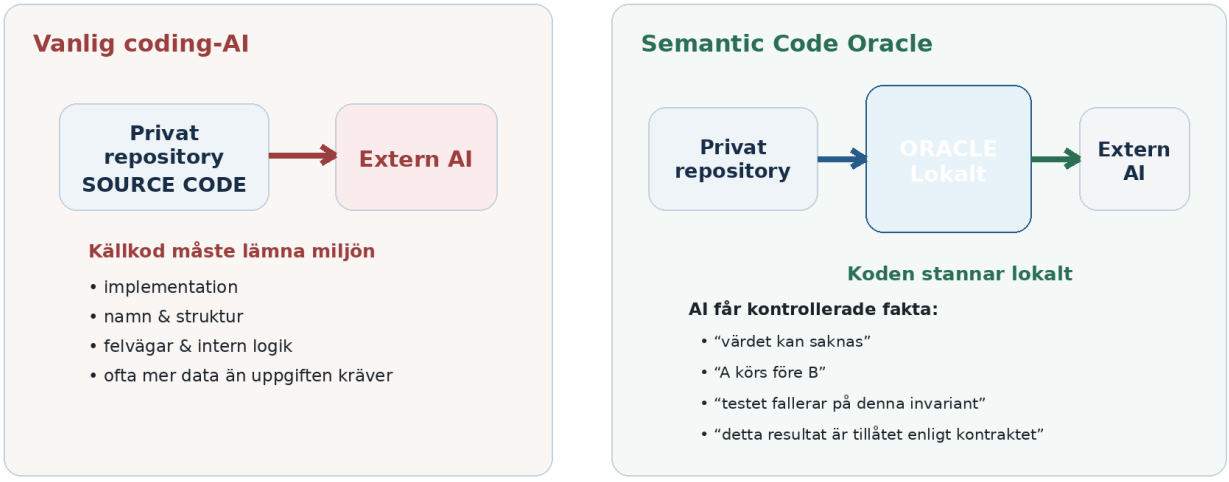
SEMANTIC CODE ORACLE

Extern AI mot privat och hemlig kod

Hur en AI kan förstå, felsöka och förändra program utan direkt source-access

Från “AI läser koden” till “AI frågar om koden”



Grundprincipen: flytta gränssnittet från source code till kontrollerade semantic facts.

Kärnbudskap

Koden stannar i den skyddade miljön. AI:n frågar Oracle om fakta, föreslår semantiska förändringar och Oracle patchar samt verifierar lokalt. För användning med hemlig eller särskilt skyddsvärd kod måste även de fakta som lämnar miljön bedömas och frisläppas enligt policy.

1. Problemet vi försöker lösa

Dagens coding-agenter bygger normalt sin förståelse genom att läsa filer, söka i repositoryt, skicka kod till en modell och sedan skriva tillbaka kod. Det fungerar bra i vanliga utvecklingsmiljöer, men skapar ett grundproblem när implementationen är affärskritisk, exportkontrollerad, säkerhetskänslig, privat eller hemlig.

- En extern AI får ofta mer information än den specifika uppgiften kräver.
- Source code avslöjar inte bara algoritmer utan även arkitektur, interna namn, säkerhetslogik, beroenden och felvägar.
- Att maska enstaka hemligheter löser inte problemet om hela implementationen i sig är skyddsvärd.
- För verksamhet med hemlig eller särskilt skyddsvärd information är frågan inte bara dataskydd utan vilken information som över huvud taget får lämna säkerhetsdomänen.

Oracle-idén

Semantic Code Oracle vänder på modellen. AI:n får inte repository-access. I stället får den ett smalt MCP-gränssnitt där den kan fråga om programmet: typer, kontrollflöde, dataflöde, kontrakt, runtime-beteende och verifieringsresultat. Oracle läser koden lokalt och returnerar så lite information som möjligt för att uppgiften ska kunna lösas.

Enkel formulering

Cursor frågar om koden. Oracle svarar om koden. Koden själv stannar lokalt.

2. Så fungerar en utvecklingsuppgift steg för steg



Samma modell kan användas för debugging, senare feature work och greenfield-konstruktion.

Vad som är annorlunda

AI:n skickar inte tillbaka en hel fil. Den uttrycker ett semantiskt intent, till exempel att en validering måste ske före en side effect, att ett Promise-resultat måste inväntas eller att ett visst argument ska komma från en annan data-origin. Oracle översätter intentet till en lokal AST/semantic patch och kontrollerar att förändringen är säker.

- Källkoden används lokalt som analysmaterial, inte som modell-input.
- Patchen appliceras innanför trust boundaryn.
- Kompilering, tester, runtime-probes och invariants körs lokalt.
- Vid osäker eller strukturellt farlig patch kan Oracle avvisa eller rollbacka.
- Extern AI kan fortsätta resonera på resultatet utan att få den privata implementationen.

3. Vad AI:n får se - och vad som stannar lokalt

Stannar i skyddad miljö	Kan lämnas ut efter policy
Repository och filinnehåll	Boolean/semantic fact: “värdet kan saknas”
Privata implementationer	Kontrollerad signatur eller typinformation
Build/test/runtime-miljö	Dataflow-relation: “värdet kommer från operation X”
Credentials, config och skyddade moduler	Control-flow-relation: “A sker före B”
Konkreta patchar på disk	Verifieringsresultat: PASS / FAIL / UNKNOWN
Hidden ground truth	Sanitiserad runtime-evidens när policyn tillåter det

Viktig säkerhetsgräns

“0 bytes source” betyder inte automatiskt “0 hemlig information”. Symbolnamn, arkitektur, runtime-värden och kombinationer av många små semantic facts kan i sig vara skyddsvärda. Därför måste high-assurance-versionen bedöma och godkänna även Oracles svar.

4. MCP-verktygen - Oracles säkra arbetsyta

MCP-verktygen är inte “AI-funktioner” i sig. De är ett begränsat API som gör att en extern agent kan undersöka och förändra ett privat program utan att få fria fil- eller shell-rättigheter.

Verktyg	Enkel beskrivning
oracle_search_symbols	Hittar klasser, funktioner och metoder utan att läsa privata filer.
signatur/struktur-verktyg	Visar parametrar, returtyper, relationer och strukturella fakta.
oracle_ask_fact	Ställer en generell semantisk fråga om programmet.
oracle_expression_fact	Förklarar värde-origin, uttrycksroll och hur ett värde används.
oracle_control_flow_fact	Visar branches, ordning, dominans och vilka vägar som kan köras.
oracle_dataflow_fact	Följer data från källa till variabel, argument, returvärde eller side effect.
oracle_get_contract	Visar tillåtna resultat, states och domänkontrakt som Oracle kan härleda.
oracle_get_diagnostics	Returnerar compiler/type-diagnostics utan att exponera source.
oracle_trace_execution	Kör ett lokalt scenario och returnerar en sanitiserad execution trace.
oracle_get_runtime_snapshot	Ger relevanta runtime-fakta vid en viss punkt, enligt disclosure policy.
oracle_generate_probe	Skapar och kör en lokal probe för en specifik hypotes.
oracle_compare_behavior	Jämför två beteenden eller före/efter patch.
oracle_check_invariant	Kontrollerar en semantisk regel, t.ex. att en side effect aldrig sker på en nekad väg.
oracle_run_tests	Kör lokala tester och returnerar kontrollerade resultat.
oracle_get_change_history	Visar relevant förändringshistorik utan fri repo-access.

5. MCP-verktyg för patchning, recovery och capability discovery

Verktyg	Enkel beskrivning
oracle_investigate	Samlar flera evidenskällor till en strukturerad undersökning.
oracle_get_investigation_state	Visar vad sessionen redan vet och på vilken repository-revision.
oracle_get_capabilities	Visar vilka semantiska patchoperationer Oracle kan uttrycka.
oracle_propose_patch	Föreslår en semantisk, lokal kodförändring.
oracle_propose_patch_plan	Kör flera semantiska operationer som en atomisk staged plan.
dry-run-plan / capability blocker	Testar om en plan går att uttrycka säkert innan repositoryt ändras.
oracle_verify_patch_persisted	Verifierar att patchen verkligen finns i den auktoritativa lokala workspacen.
oracle_list_revisions	Visar sessionens lokala revisionshistorik.
oracle_restore_revision	Rollbackar till en tidigare säker revision.

Exempel på patch-intent

Istället för rå source-edit

“Se till att precondition-regionen exekveras före side effect-regionen och bevara dependent guards.” Oracle avgör lokalt vilka statements som hör ihop, kontrollerar dependencies, applicerar patchen och verifierar beteendet.

6. Vad vi har bevisat - och vad vi inte har bevisat ännu

Observerad evidens hittills

- I den rena Oracle v2-benchmarken löstes 7 av 10 blinda TypeScript-buggar.
- Rå source-exponering var 0 bytes i samtliga benchmark-cases, och protected leakage var 0 bytes.
- Oracle har kunnat diagnostisera problem inom nullable values, async, call order, stale state, wrong arguments, auth/side effects, exception mapping och branch mutations.
- Systemet har transactional patching, revisions, rollback, runtime evidence och hidden semantic evaluation.

Viktig aktuell begränsning

Efter v2.1 identifierades ett konkret gap i ett regressionstest: en korrekt call-order-repair krävde att en hel dependency-region (producer + dependent guard) flyttades tillsammans. Det betyder att “operationen reorder finns” inte är samma sak som att varje realistisk dependency-aware reorder är uttryckbar. Det gapet bör stängas innan nya blind holdout-cases används som huvudbevis.

Rätt slutsats idag

Konceptet har stark teknisk signal och flera fungerande blind-reparationer, men vi har ännu inte tillräcklig evidens för att säga att Oracle fungerar för generell mjukvaruutveckling eller att en extern AI-lösning är godkänd för drift med hemlig eller särskilt skyddsvärd information.

7. Hur vi bevisar att Oracle fungerar för utveckling

För att gå från “lovande prototyp” till ett trovärdigt utvecklingspåstående behöver evidensen breddas i flera steg.

Nivå	Test	Vad det visar	Exempel på evidens
1	5-10 nya blinda buggar	Generaliserar debugging?	Solved rate, queries, revisions, 0 source
2	Greenfield API/app	Kan AI designa och bygga ny kod genom Oracle?	Acceptance tests, architecture, iteration
3	Java vertical slice + blind cases	Är modellen språkoberoende?	Samma MCP/IR över TS + Java
4	30-50 unseen development tasks	Fungerar bredare utvecklingsarbete?	Features, refactors, API, security, state
5	Riktiga repos / utvecklare	Praktisk användbarhet	Review acceptance, time, interventions
6	Adversarial security evaluation	Håller disclosure-modellen?	Cumulative inference, bypass, leakage

För en första generaliseringssignal är 4/5 eller 5/5 på helt nya, realistiska blind-cases stark evidens för teknisk feasibility. För påståendet “fungerar för utveckling” behövs därefter större task-bredd, flera repos och helst flera språk.

8. Greenfield och Java - varför de är viktiga

Greenfield

Debugging börjar med befintlig kod. Greenfield testar något svårare: kan agenten beskriva arkitektur, nya typer, funktioner, routes och tester medan Oracle skapar den konkreta implementationen lokalt? Det kräver construction capabilities som create_module, create_interface, create_function, add_route och create_test - fortfarande utan att ge AI:n fri source-editor.

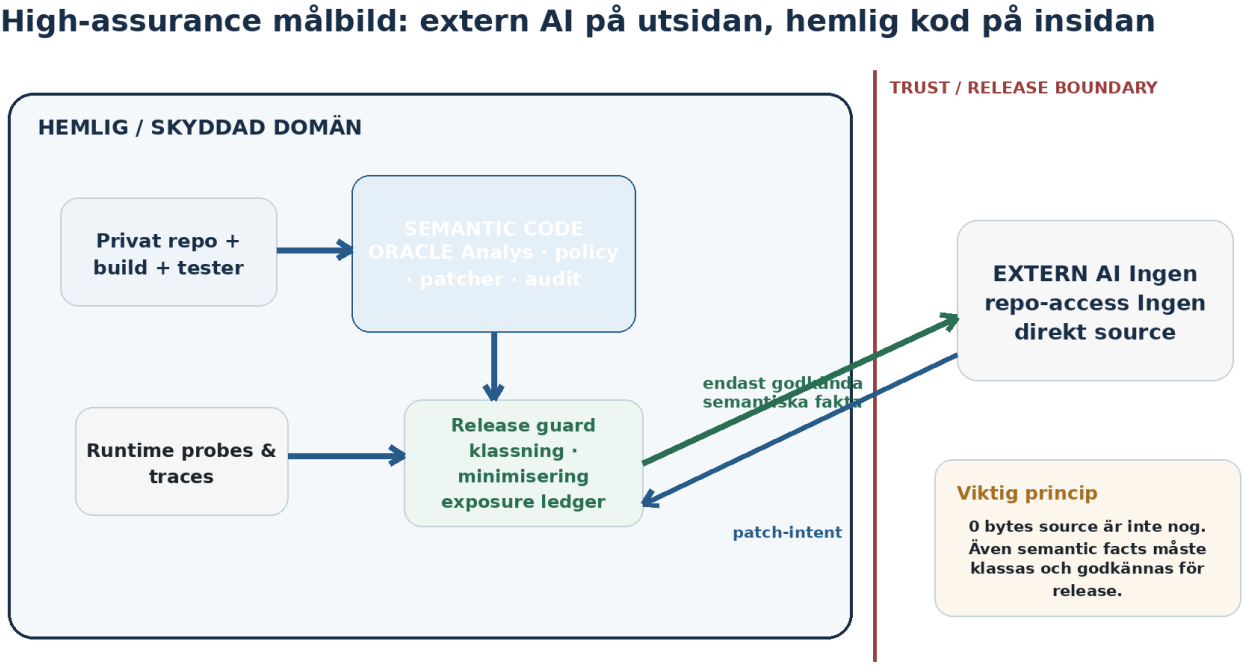
Java

Java testar om arkitekturen verkligen ligger ovanför programmeringsspråket. Målet är att samma MCP-frågor och samma SemanticPatch IR ska fungera mot flera ProgramAnalysisAdapters.

Målarkitektur

External AI -> samma Oracle MCP -> ProgramAnalysisAdapter -> TypeScript/ts-morph eller Java/Spoon/JDT -> lokal patch + verifiering.

9. För hemlig kod krävs en information release guard



Den externa AI:n bör behandlas som lägre trust. Oracle måste avgöra vad som får passera gränsen.

Det säkraste målet är inte att göra en extern AI “behörig att läsa hemlig kod”. Målet är att den externa AI:n aldrig behöver få den hemliga implementationen - och helst inte heller andra hemliga eller skyddsvärda fakta. Oracle blir då en cross-domain-liknande release-mekanism som bedömer, minimerar och loggar varje informationsutlämning.

Tekniska kontroller som behövs

- Klassning av varje semantic fact och runtime-observation.
- Deterministisk allow/deny-policy som inte kan överridas av modellen.
- Opaque identifiers eller abstraktion när verkliga symbolnamn inte behöver lämnas ut.
- Exposure ledger per session och skydd mot kumulativ inference.
- Disclosure budgets och progressive disclosure: boolean fact -> relation -> sanitiserad struktur -> endast i sista hand minimal source.
- Audit trail över frågor, svar, patch-intent, policybeslut och revisionshistorik.
- Lokal testning, patchning, rollback och verifiering innanför den skyddade miljön.
- Provider-isolering: extern AI ska inte ha shell, filesystem eller direkt HTTP till den privata miljön.

10. Formella förutsättningar i svensk säkerhetsskyddskontext

Detta avsnitt är en teknisk sammanfattning, inte ett myndighetsbeslut eller juridiskt godkännande. Den faktiska bedömningen måste göras av ansvarig verksamhetsutövare och berörd tillsynsmyndighet.

- Säkerhetsskyddsanalys: verksamhetsutövaren ska identifiera skyddsvärden, hot, sårbarheter och nödvändiga säkerhetsskyddsåtgärder.
- Särskild säkerhetsskyddsbedömning inför driftsättning av informationssystem som har betydelse för säkerhetskänslig verksamhet.
- Systemgodkännande: informationssystem får inte tas i drift i säkerhetskänslig verksamhet innan de godkänts från säkerhetsskyddssynpunkt av verksamhetsutövaren.
- För system som kan förutses behandla uppgifter i säkerhetsskyddsklassen konfidentiell eller högre finns krav på skriftligt samråd med Säkerhetspolisen eller, inom relevant tillsynsområde, Försvarsmakten före drift eller väsentlig förändring.
- Systemet ska ha skydd mot obehörig åtkomst, avlyssning och skadlig påverkan samt spårbarhet för säkerhetsrelevanta händelser.
- Om uppgifter som omfattas av säkerhetsskydd behandlas i ett informationssystem utanför verksamhetsutövarens kontroll måste verksamhetsutövaren försäkra sig om att skyddet är tillräckligt. Om hemliga eller andra skyddsvärda uppgifter faktiskt kommuniceras utanför kontrollen tillkommer särskilda krav på skydd, inklusive regler kring godkända kryptografiska funktioner.

Konsekvens för Oracle

Om Oracle kan bevisa att endast information som är tillåten för den externa AI-domänen lämnar säkerhetsgränsen, är målet att hålla AI-tjänsten utanför behandling av hemlig eller skyddsvärd information. Om hemliga eller skyddsvärda facts faktiskt skickas till tjänsten förändras förutsättningarna väsentligt och hela mottagarsystemet/leverantörskedjan måste bedömas därefter.

11. Risker som måste lösas - även utan source disclosure

Risk	Exempel	Motåtgärd
Kumulativ inference	Många små svar avslöjar tillsammans arkitekturen	Exposure ledger, session budget, composition policy
Symbol leakage	Namn avslöjar projekt, system eller förmåga	Opaque IDs, aliasing, least disclosure
Runtime leakage	Värden/loggar innehåller hemlig data	Sanitization, schemas, allowlisted fields
Capability abuse	Agenten försöker få mer data via ovanliga probes	Deterministisk policy, rate/budget, query classes
Patch abuse	Semantiskt intent ger farlig transformation	Staging, dependency checks, dry-run, transactional commit
Supply-chain risk	Oracle/provider/dependencies manipuleras	Signed builds, SBOM, controlled updates, reproducible builds
Provider retention	Extern tjänst lagrar prompts/metadata	Kontrakt, provider controls - eller håll all hemlig data utanför providern

12. Roadmap till high-assurance användning



Teknisk funktion och säkerhetsgodkännande är två parallella evidenskedjor.

Nästa konkreta steg

1. Stäng det identifierade dependency-aware region-reorder-gapet och regressionstesta det.

2. Kör 5 nya realistiska blind-cases utan Oracle-ändringar mellan fallen.

3. Bygg ett greenfield-benchmark där AI:n skapar ett litet API/app genom construction tools.

4. Bygg en Java ProgramAnalysisAdapter och tre smoke cases innan större Java-benchmark.

5. Frys Oracle och jämför RuleBasedDecisionModel mot Laya på samma unseen holdout.

6. Bygg high-assurance disclosure: fact classification, cumulative inference controls, audit och policy proofs.

7. Genomför separat säkerhetsskyddsanalys, systembedömning, eventuellt samråd och formellt systemgodkännande inför verklig säkerhetskänslig drift.

13. Vad vi kan säga idag

Starkt och försvarbart påstående

Semantic Code Oracle har demonstrerat att en extern coding-agent kan diagnostisera och reparera flera blinda TypeScript-fel utan direkt repository-access och med 0 bytes rå source i de uppmätta benchmarkkörningarna.

För tidigt att påstå

“Oracle fungerar för all mjukvaruutveckling” eller “lösningen är godkänd för drift med hemlig information”. Det kräver bredare development-evidens, fler språk, adversarial disclosure-testing samt verksamhetens formella säkerhetsskyddsprocess.

Visionen i en mening

Gör koden onödig att läsa - inte bara svårare att komma åt.

14. Officiella källor och begrepp

Säkerhetsdelen ovan har kontrollerats mot aktuella officiella svenska källor. Exakta krav beror på verksamhet, tillsynsområde, säkerhetsskyddsklass, systemarkitektur och vilken information som faktiskt lämnar verksamhetsutövarens kontroll.

Säkerhetsskyddsförordning (2021:955), Sveriges riksdag
Särskilt 2 kap. om säkerhetsskyddsanalys och 3 kap. om informationssystem, samråd, systemgodkännande, skyddsåtgärder och system utanför verksamhetsutövarens kontroll. (riksdagen.se)

Säkerhetsskyddslag (2018:585), Sveriges riksdag
Grundläggande krav på informationssäkerhet i säkerhetskänslig verksamhet. (riksdagen.se)

Säkerhetspolisen - Vägledning i säkerhetsskydd: Informationssäkerhet
Vägledning om bl.a. godkännande av informationssystem som hanterar uppgifter som omfattas av säkerhetsskydd. (sakerhetspolisen.se)

Försvarsmakten - Reglemente Säkerhetstjänst 2021
Vägledande material om bl.a. behandling/kommunikation utanför verksamhetsutövarens kontroll. (forsvarsmakten.se)

Begrepp

Begrepp	Betydelse
Semantic fact	En abstraherad uppgift om programmet, t.ex. en typrelation, call order eller verifierad invariant.
Trust boundary	Gräns där information eller kontroll går mellan olika säkerhets-/förtroendedomäner.
Release guard	Komponent som avgör vilken information som får passera från högre till lägre trust.
Exposure ledger	Sessionhistorik över vad som redan har lämnats ut, för att kunna bedöma kumulativ informationsrisk.
Semantic patch	En kodförändring uttryckt som intent/struktur snarare än som rå textredigering.
Hidden evaluator	Oberoende lokal evaluator som avgör om affärsbeteendet faktiskt är korrekt utan att ge facit till agenten.